CLASSIFIED FLIGHT MANUAL // REV 2026.10

CHASMA: SURVIVE THE HOLLOWSTAR

A Complete Visual Architecture, Newtonian Physics & Neural AI Infographic of the Codebase

2,374	22	25.6 KB	< 50 us	0 Files	50,000
TOTAL PYTHON LOC	ACTIVE .PY MODULES	ONNX BRAIN (L1 CACHE)	CPU AI INFERENCE	EXTERNAL AUDIO/SPRITES	HEADLESS TRAIN FPS

1. 1988 AMIGA HERITAGE & CORE PREMISE

EBONSTAR HOMAGE

Inspired by the classic 1988 Commodore Amiga title Ebonstar (by Robert 'Wolf' McNally / The Dreamers Guild), Chasma reinvents 4-player couch dogfighting by eliminating health bars, shields, and attrition damage.

THE ZERO-DAMAGE KINETIC KNOCKBACK PARADIGM

Blasters deal ZERO HP damage. Instead, every shot imparts heavy kinetic knockback (v = 5.0), and ship collisions transfer elastic momentum (e = 0.8). Victory requires banking shots off walls and shoving opponents across the event horizon of the wandering central singularity: The Hollowstar!

FUN CODEBASE FACTS & ENGINEERING HIGHLIGHTS:

* Secret Prototype Name: .idea/Space Game.iml reveals Chasma started as 'Space Game'!
* 100% Procedural Everything: Zero .png or .wav files! Every hull, flame, warped spacetime grid vertex, and all 8 synth sounds are generated from pure math.
* Instant AI Backfill: Connect 0 to 4 controllers; empty slots are seamlessly piloted by a 25.6 KB ONNX neural network trained via 250k steps of RL.

2. MODERN PYTHON TECH STACK

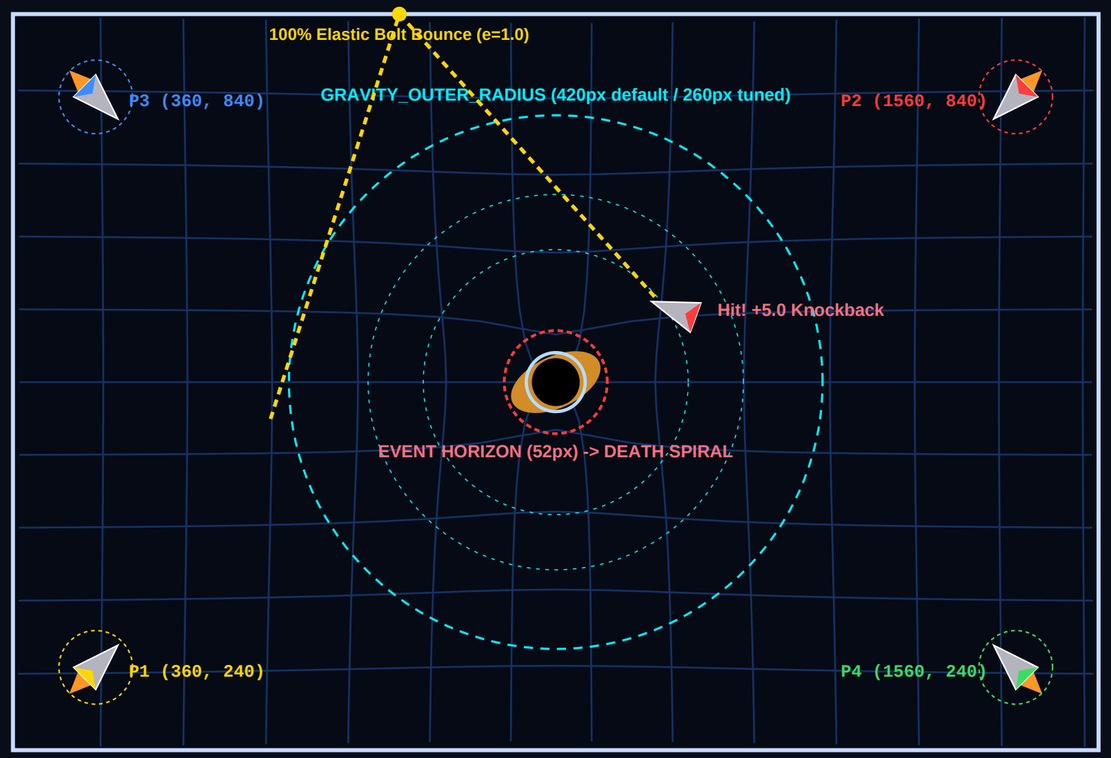
PYTHON 3.10-3.13

LAYER	PACKAGE	ARCHITECTURAL ROLE
2D Engine & GL	`arcade >= 3.0.0`	OpenGL batch shapes, 60Hz loop, scene Views
Procedural Audio	`pyglet.media`	Real-time waveform synthesis & ADSR envelopes
Neural Runtime	`onnxruntime >= 1.18`	C++ sub-50us CPU policy inference (Opset 18)
RL Environment	`gymnasium >= 1.0`	Headless 50,000 FPS training arena simulation
Policy Trainer	`stable-baselines3`	PPO Actor-Critic optimization & ONNX export
Graph Tracing	`torch >= 2.2.0`	MLP [64, 64] actor graph export to ONNX

3. TACTICAL ARENA BLUEPRINT (1920 x 1080 LETTERBOXED TO 1400 x 800)

64px WARPED SPACETIME

WINDOW: 1920 x 1080 FULL HD | ARENA_MARGIN_X = 260px | ARENA_MARGIN_Y = 140px



ARENA BOUNDS: X in [260, 1660] (1400px) | Y in [140, 940] (800px) | Ship Wall Bounce = 0.5x

CLEAN 5-PACKAGE ARCHITECTURE & 60 HZ ORCHESTRATION

How config, core, entities, systems, views, and ai interlock with zero circular dependencies

1. PACKAGE TOPOLOGY & MODULE DEPENDENCY GRAPH (22 PYTHON FILES)

2,374 TOTAL LOC

main.py (10 LOC)
ChasmaGame() -> TitleView() -> arcade.run()

chasma.core

game.py (55 LOC)
ChasmaGame(arcade.Window)
Controller init & F1 hook
arena.py (93 LOC)
Arena boundary clamping
12-seg warped spacetime

chasma.views (Scene Graph)

title_view.py (50 LOC)
game_view.py (304 LOC)
60Hz orchestrator, gravity,
ship collisions, AI backfill
pause_view.py (65 LOC)
game_over_view.py (55 LOC)

chasma.ai (RL & ONNX)

ai_controller.py (257 LOC)
14-float obs + ORT + heuristic
training_env.py (335 LOC)
ChasmaArenaEnv & HeadlessShip
train.py (121 LOC)
PPO -> chasma_pilot.onnx (25KB)

chasma.entities (Physics Domain Models)

base_entity.py (31 LOC) -> BaseEntity
center_x/y, velocity_x/y, angle, ShapeElementList
player.py (193 LOC) -> Player(BaseEntity)
Dual-triangle mesh, thrust flame, rising-edge fire,
death spiral state machine & respawn countdown
hollowstar.py (81 LOC) -> Hollowstar(BaseEntity)
Brownian wander, center pull, 4-tier accretion disc
projectile.py (72 LOC) -> Projectile(BaseEntity)
8.0 px/f bolt, 120f lifetime, 24px circular hit check

chasma.systems (Low-Level Engine Services)

input_manager.py (89 LOC) -> InputManager
Unifies 1-4 gamepads + keyboard fallback into InputState
Applies STICK_DEADZONE (0.2) & TRIGGER_DEADZONE (0.05)
sound.py (96 LOC) -> SoundManager
Synthesizes 8 retro waveforms in RAM via pyglet.media
debug_tuner.py (138 LOC) -> DebugTuner
F1 interactive HUD, live gravity rings & JSON persistence

chasma.config (Centralized Constants & Live-Tuning Overlay Engine)

settings.py (80 LOC): Screen (1920x1080), Arena margins, Ship physics, Gravity & Blaster constants
colors.py (23 LOC): ChasmaColors palette (SPACE_BLACK, GRID_BLUE, HOLLOWSTAR_DISC, P1-P4 colors)
controls.py (7 LOC): STICK_DEADZONE = 0.2, TRIGGER_DEADZONE = 0.05
tuning.py (92 LOC): Overlays settings.py with tuning.overrides.json; hot-mutates globals in-place!

2. THE 60 FPS FRAME PIPELINE (GameView.on_update)

16.67 ms BUDGET

1. Input & Edge-Pause Poll
InputManager.get_states() polls pads/keys; check_any_pause()

2. Neural AI Pilot Backfill
AIController builds 14-float vector -> ONNX (<50us) -> InputState

3. Ship & Hollowstar Motion
Integrate ship thrust & 0.99 drag; Hollowstar wander + center pull

4. Gravity & Spiral Capture
Inverse-square pull inside R_outer; start_spiral() if d < 52px

5. Elastic Ship Collisions
Resolve circle overlap + elastic impulse (e=0.8) & 'hit' sound

6. Blaster Bolt Simulation
Spawn bolts on rising-edge fire; circular hit (24px) -> +5.0 knockback

7. Arena Boundary Rebounds
Dampen ship wall bounces (0.5x); reflect bolts & Hollowstar (1.0x)

8. Match Victory Arbitration
Count ships with lives > 0; transition to GameOverView when <= 1

3. SCENE VIEW & SHIP LIFECYCLE STATE MACHINES

ARCADE.VIEW

TitleView
Press ENTER/START

GameView
4-Ship Arena Loop

PauseView
ESC / START Modal

GameOverView
<= 1 Ship Alive

PLAYER SHIP LIFECYCLE (3 LIVES):
Active -> Crosses 52px -> Spiraling (50f @ 15 deg/f) -> Dead (90f delay) -> Respawn

NEWTONIAN INERTIA, SPACETIME WARPING & ELASTIC MOMENTUM

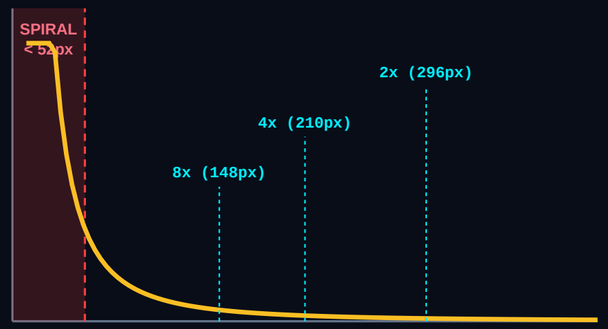
Exact mathematical equations powering the Hollowstar singularity, relativistic grid distortion, and kinetic knockback

1. SINGULARITY & DEATH SPIRAL

INVERSE-SQUARE

The Hollowstar drifts at 0.9 px/frame with Brownian heading noise (-0.03 to +0.03 rad/frame) plus a gentle centering spring force (* 0.005).

```
clamped_d = max(dist, GRAVITY_MIN_DIST)
force = GRAVITY_STRENGTH / (clamped_d ** 2)
v += (delta_p / dist) * force
```



Distance from Hollowstar Center (px) -> R_outer = 420px

THE SPIRAL OF DEATH SEQUENCE (d < 52px):

1. Lock Controls: player.spiraling = True; triggers 90 Hz 'suck' synth.

2. Rapid Spin: Ship rotates uncontrollably at SPIRAL_SPIN_SPEED (15 deg/f).

3. Vortex Pull: Accelerates inward at 8.0 * THRUST_FORCE (1.2 px/f^2).

4. Speed Clamp: Velocity clamped to MAX_SPIRAL_SPEED = 10.0 (no overshoot).

5. Consumption: After SPIRAL_DURATION (50f), ship dies (-1 life, 'death' noise).

6. Respawn Delay: Waits RESPAWN_DELAY = 90 frames (1.5s) before corner reset.

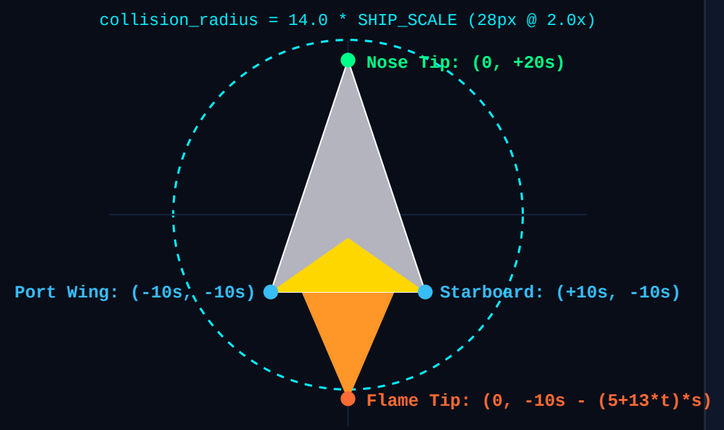
2. SPACETIME WARP & SHIP MESH

PROCEDURAL OPENGL

Every 64px grid line is split into 12 segments.
Each vertex (px, py) is pulled toward the Hollowstar to create a real-time relativistic gravity funnel:

```
warp = GRID_WARP_STRENGTH / (d + FALLOFF)
warp = min(d * 0.8, 1800 / (d + 60))
p_warped = p - ((p - h) / d) * warp
```

PROCEDURAL SHIP POLYGON & COLLISION SPHERE



COMPASS HEADING & NEWTONIAN DRAG:

In Chasma, 0 deg points North (+Y) & 90 deg points East (+X):

```
rad = math.radians(self.angle)
self.velocity_x += math.sin(rad) * self.thrust
self.velocity_y += math.cos(rad) * self.thrust
self.velocity_x *= DRAG (0.99)  # capped at MAX_SPEED=7.0
```

Rising-edge fire latch (_fire_held) prevents holding Fire from auto-spamming bolts; enforces 45-frame cooldown.

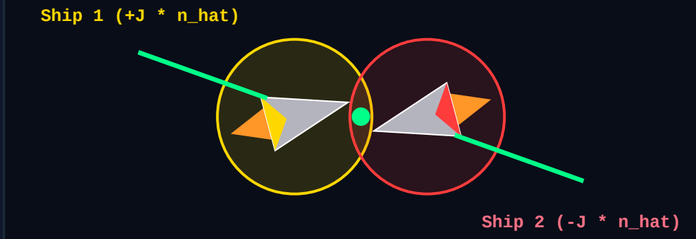
3. ELASTIC IMPULSE & KNOCKBACK

MOMENTUM TRANSFER

When two ships collide (dist < r1 + r2), GameView executes a 2-phase rigid-body impulse solver with restitution e = 0.8 and a minimum bump floor:

```
1. Push Apart: p1 += n*(ov/2); p2 -= n*(ov/2)
2. Impulse: j = -(1 + 0.8) * (v_rel . n) * 0.5
3. Total:   J = (j + 1.0) * COLLISION_STRENGTH
```

ELASTIC SHIP-TO-SHIP COLLISION (e = 0.8)



Minimum bump (1.0) ensures stationary ships still separate!

BLASTER BOLT KINETICS (projectile.py):

Bolt Velocity	BLASTER_SPEED = 8.0 px/frame
Bolt Lifetime	BLASTER_LIFETIME = 120 frames (2.0s)
Fire Cooldown	BLASTER_COOLDOWN = 45 frames (0.75s)
Visual Geometry	20px line segment, 5px width, owner color
Wall Rebound	100% reflection (e=1.0) + atan2 angle update
Hit Radius	24px circular check (ignores bolt owner)
Knockback Force	+5.0 velocity impulse along bolt heading

INSIDE THE 25.6 KB NEURAL PILOT: 14 FLOATS -> 3 ACTION HEADS

How AIController encodes ego-centric arena telemetry into L1-cache-resident ONNX inference in under 50 microseconds

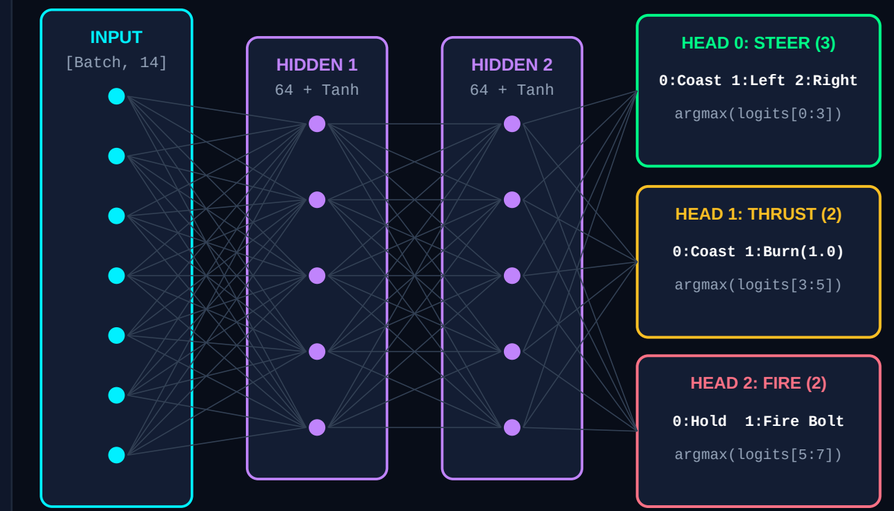
1. THE 14-FLOAT EGO-CENTRIC OBSERVATION VECTOR (dtype=float32)

NORMALIZED [-1, 1]

IDX	FEATURE KEY	EXACT FORMULA	RANGE	TACTICAL PURPOSE
[0]	self_vx	clip(bot.vx / 10.0, -1, 1)	[-1, 1]	Own horizontal drift & momentum
[1]	self_vy	clip(bot.vy / 10.0, -1, 1)	[-1, 1]	Own vertical drift & momentum
[2]	sin_angle	sin(radians(bot.angle))	[-1, 1]	Continuous X nose unit vector
[3]	cos_angle	cos(radians(bot.angle))	[-1, 1]	Continuous Y nose unit vector
[4]	rel_target_x	clip((tx - bx) / 1920, -1, 1)	[-1, 1]	Relative X offset to nearest foe
[5]	rel_target_y	clip((ty - by) / 1080, -1, 1)	[-1, 1]	Relative Y offset to nearest foe
[6]	target_dist	clip(dist / 2202.9, 0, 1)	[0, 1]	Proximity to target (sweet spot)
[7]	rel_target_vx	clip((tvx - bvx) / 10, -1, 1)	[-1, 1]	Relative X closing/fleeing speed
[8]	rel_target_vy	clip((tvy - bvy) / 10, -1, 1)	[-1, 1]	Relative Y closing/fleeing speed
[9]	heading_align	H . D = sin*Dx + cos*Dy	[-1, 1]	**Dot Product (cos a): +1=DEAD AIM!**
[10]	rel_hs_x	clip((hx - bx) / 1920, -1, 1)	[-1, 1]	Relative X offset to Hollowstar
[11]	rel_hs_y	clip((hy - by) / 1080, -1, 1)	[-1, 1]	Relative Y offset to Hollowstar
[12]	hs_dist	clip(hs_dist / 2202.9, 0, 1)	[0, 1]	Normalized distance to singularity
[13]	in_grav_zone	1.0 if d < R_outer else 0.0	{0, 1}	Binary emergency gravity alarm

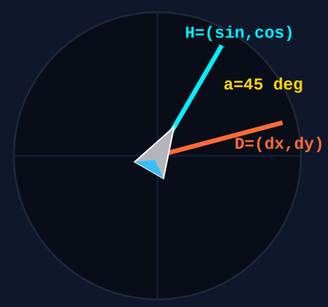
2. MLP [64, 64] ACTOR NETWORK & 3 HEADS

OPSET 18 / 25.6 KB



3. VECTOR MATH SECRET SAUCE & HEURISTIC FALLBACK

DOT & CROSS PRODUCT



1. Continuous Unit Circle (sin, cos):
Eliminates the 359 deg -> 1 deg wrap-around cliff that confuses standard MLP networks.

2. Dot Product Aiming (H . D = cos a):
+1.0 = Dead Aim (Fire!) | 0.0 = 90 deg Flank
-1.0 = Target directly behind ship.

3. 2D Cross Product (H x D = sin a):
Used by _predict_heuristic() fallback pilot:
cross > 0 -> Turn Right | cross < 0 -> Left!

FALLBACK PILOT (_predict_heuristic):
Combines Target Pursuit + Hollowstar Repulsion + 120px Wall Buffer!

REINFORCEMENT LEARNING // GYMNASIUM + PPO

THE 50,000 FPS HEADLESS FLIGHT ACADEMY & REWARD SHAPING

How ChasmaArenaEnv cures 'Turret Camping' and trains aggressive orbital dogfighters in 250,000 timesteps

1. ANTI-CAMPING REWARD FUNCTION (chasma/ai/training_env.py)

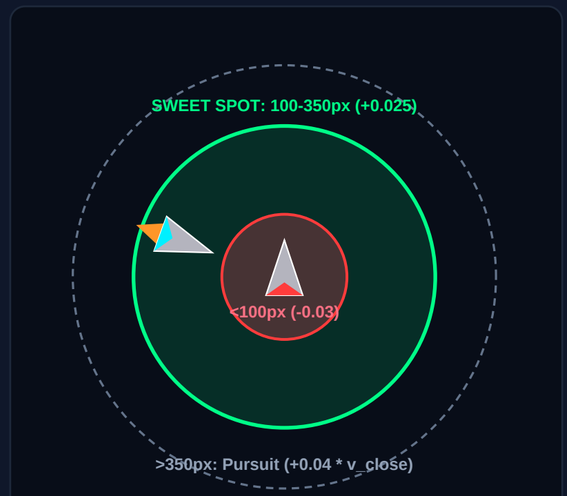
COMMIT 6601e0f

Early RL agents discovered a lazy local optimum: park stationary in a corner, spin like a turret, and farm aiming points. The v2 reward function enforces active dogfighting:

REWARD / PENALTY TERM	FORMULA / MAGNITUDE	BEHAVIORAL SHAPING EFFECT
Blaster Hit Impact	+10.0 per hit	Primary offensive jackpot; rewards lead-shot accuracy
Singularity Death	-15.0 (Terminal)	Strong survival instinct around Hollowstar (<52px)
Gravity Zone Tax	-0.02 * (1 - d/R_out)	Continuous gradient pushing bot out of vortex ring
Moving Aim Bonus	+0.04 * max(0, cos a)	ONLY granted if speed >= 1.2! Zero reward if parked!
Camping Penalty	-0.03 * (1.2 - speed)	Directly punishes hovering or stationary turret spinning
Cruise Speed Bonus	+0.02 (1.8 <= v <= 5.2)	Rewards smooth, controllable orbital strafing speed
Sweet-Spot Range	+0.025 (100 <= d <= 350)	Maintains ideal dogfight ring around opponent
Anti-Ramming Tax	-0.03 (dist < 100px)	Prevents reckless kamikaze collisions into enemy
Pursuit Closing	+0.04 * (closing_v / 6)	When far (d > 350px), rewards closing the gap
Throttle Discipline	-0.02 (v>6 or cos<-0.3)	Penalizes thrusting while overspeeding or facing away

2. TACTICAL ENGAGEMENT SWEET-SPOT & VELOCITY GATES

REWARD TOPOLOGY



SHIP SPEED REWARD GATES:

v < 1.2 px/f (Stagnant / Parked)
`Aim Bonus = 0! Penalty: -0.03*(1.2-v)`

v >= 1.2 px/f (Active Motion)
`Unlocks +0.04 * cos(a) Aim Bonus!`

1.8 <= v <= 5.2 px/f (Cruise)
`Optimal dogfight speed (+0.02/frame)`

v > 6.0 px/f + Burning Thrust
`Overspeed reckless penalty (-0.02)`

cos(a) < -0.3 + Burning Thrust
`Wrong-way thrust penalty (-0.02)`

3. PPO HYPERPARAMETERS & ONNX EXPORT (chasma/ai/train.py)

250,000 TIMESTEPS

RL Algorithm:	PPO (Proximal Policy Optimization)
Policy Architecture:	MlpPolicy net_arch=[64, 64] (Tanh)
Learning Rate:	3e-4 (Adam optimizer)
Rollout Buffer:	n_steps = 2048 | batch_size = 64
PPO Epochs & Clip:	n_epochs = 10 | clip_range = 0.20
Discount & GAE:	gamma = 0.99 | gae_lambda = 0.95
Entropy Coef:	ent_coef = 0.01 (encourages exploration)
Episode Horizon:	1,800 ticks (30 seconds @ 60 FPS)

```
class OnnxablePolicy(th.nn.Module):  # Strips Critic head & exports pure Actor
    features = self.policy.extract_features(obs, self.policy.pi_features_extractor)
    logits = self.policy.action_net(self.policy.mlp_extractor.forward_actor(features))
    return th.stack([argmax(logits[:,0:3]), argmax(logits[:,3:5]), argmax(logits[:,5:7])], dim=1)
```

ZERO-ASSET PROCEDURAL SYNTHESIZER, F1 LIVE TUNER & CONTROLS

Real-time waveform synthesis (sound.py), interactive physics hot-reloading (debug_tuner.py), and multi-controller mapping

1. THE 8 PROCEDURAL SYNTHESIZER PATCHES (chasma/systems/sound.py)

0 WAV FILES

At startup, SoundManager._generate_sounds() synthesizes 8 retro sound effects directly in RAM using pyglet.media.synthesis oscillators and envelope generators:

PATCH KEY	WAVEFORM	FREQ	DUR	ENVELOPE PROFILE	GAME EVENT
"fire"	Square	880 Hz	0.08s	LinearDecay(peak=0.30)	Blaster bolt fired
"hit"	Square	220 Hz	0.12s	LinearDecay(peak=0.40)	Blaster or ship impact
"death"	WhiteNoise	Noise	0.40s	LinearDecay(peak=0.35)	Ship consumed in core
"bounce"	Triangle	600 Hz	0.03s	LinearDecay(peak=0.15)	Ship arena wall bounce
"menu_select"	Sine	1200 Hz	0.10s	ADSR(0.01,0.05,0.04,0.3)	Menu confirmation
"game_over"	Sawtooth	110 Hz	0.80s	LinearDecay(peak=0.30)	Match defeat screen
"win"	Sine	880 Hz	0.30s	ADSR(0.02,0.10,0.15,0.3)	Victory celebration
"suck"	Sine	90 Hz	0.60s	ADSR(0.05,0.20,0.30,0.25)	Entering death spiral

GRACEFUL HEADLESS / NO-AUDIO FALLBACK:

Every oscillator is wrapped in StaticSource() and cached in self._sounds: dict[str, Source].
If audio hardware initialization fails on a headless CI server, SoundManager sets self._enabled = False and silently no-ops .play() calls without crashing the game!

2. INTERACTIVE F1 DEBUG TUNER & tuning.overrides.json

12 LIVE PARAMS

Pressing F1 opens DebugTuner, drawing live gravity contour rings and hot-saving overrides:

PARAMETER KEY	DEFAULT	ACTIVE JSON OVERRIDE	STEP (-/=)	BIG (_/+)
SHIP_SCALE	2.0	1.4 [OVERRIDDEN *]	0.1	0.5
SHIP_COLLISION_STRENGTH	1.2	1.2 (Default)	0.1	0.5
GRAVITY_STRENGTH	3200.0	2600.0 [OVERRIDDEN *]	100	500
GRAVITY_OUTER_RADIUS	420	260 [OVERRIDDEN *]	10	40
GRAVITY_INNER_RADIUS	52	52 (Default)	2	10
GRAVITY_MIN_DIST	30	30 (Default)	1	5
SPIRAL_DURATION	50	30 [OVERRIDDEN *]	2	10
SPIRAL_FORCE_MULTIPLIER	8.0	7.4 [OVERRIDDEN *]	0.2	1.0
SPIRAL_SPIN_SPEED	15	14 [OVERRIDDEN *]	0.5	2.0
MAX_SPIRAL_SPEED	10.0	10.0 (Default)	0.2	1.0
GRID_WARP_STRENGTH	1800	1800 (Default)	50	200
GRID_WARP_FALLOFF	60	60 (Default)	2	10

3. OPERATOR CONTROL DECK (InputManager & DebugTuner)

1-4 PLAYERS

GAMEPAD (P1-P4)

```
L-Stick X : Rotate (DZ=0.20)
R-Trigger : Analog Thrust
A Button  : Fire Blaster Bolt
START Btn : Edge-Triggered Pause
```

KEYBOARD (0 PADS)

```
Left/Right: Rotate (-1 / +1)
Up Arrow  : Full Thrust (1.0)
SPACEBAR  : Fire Blaster Bolt
ESC / Q   : Pause / Quit Match
```

F1 TUNER HOTKEYS

```
F1          : Toggle Overlay & Rings
[ / ]       : Cycle 12 Parameters
-/= & _/+ : Small / Big Adjust
F2/F3/F4    : Reset / Dump / Wipe
```